

Zennial Pro — AI Features Document

Table of Contents

1. Executive Summary
 2. Goals & Non-Goals
 3. Personas
 4. The TPI Score — the metric every feature ties back to
 5. Feature: Multi-Agent TPI Scoring
 6. Feature: AI Personal Tutor
 7. Feature: AI Code Mentor
 8. Feature: AI Mock Interview
 9. Feature: AI Project Reviewer
 10. Feature: Cohort Whisperer (Admin AI)
 11. Feature: Predictive Drop-off + Auto-Intervention
 12. Feature: Smart Notifications
 13. Feature: Verifiable TPI Credential
 14. Feature: Daily Commit Cadence & Code Quality as a Signal
 15. Cross-feature loops
-

1. Executive Summary

Zennial Pro is the **intelligence layer that sits on top of a training program's LMS**. The product turns thousands of candidate behavioural signals — quiz answers, page sessions, video watches, code commits — into:

- **A weekly TPI score per candidate**, broken into five dimensions
- **Real-time AI coaching** the candidate can ask anything
- **Predictive risk ranking** so admins intervene early
- **A signed, portable credential** at graduation that recruiters can verify in one click

This document goes feature-by-feature through what each AI surface does, why it exists, who uses it, and how we know it's working. It deliberately avoids implementation details — those live in code.

2. Goals & Non-Goals

Goals (the bar we hold the product to)

#	GOAL	HOW WE KNOW IT'S WORKING
G1	Make every candidate's performance measurable in real time	Every active candidate has a fresh TPI score every week, with the five dimensions visible
G2	Reduce mentor time on routine review by $\geq 50\%$	Mentor calendars show fewer hours on first-pass code reviews and mock interviews per candidate per week
G3	Reduce drop-off in pilot cohorts by $\geq 20\%$ vs. control	Week-12 completion rate, cohort-over-cohort
G4	Every graduate leaves with a portable, verifiable credential	100% of graduates have a TPI credential URL they can share
G5	Candidate data stays inside the deployment	All AI inference runs against a local LLM by default; no behavioural data leaves the box

Non-Goals (what we explicitly don't do)

- We **don't** rebuild the LMS. Course delivery, payments, attendance, quiz authoring all stay where they are.
- We **don't** write the curriculum. Customers bring their content; we score against it.
- We **don't** auto-discipline candidates. Every intervention is a coaching nudge — never punitive.
- We **don't** police candidates. We measure behaviour; we don't surveil keystrokes or webcams.

3. Personas

P1 • Candidate

- A learner inside an 8–24-week training program — typically a coding bootcamp or corporate L&D track.
- **Wants:** a clear next step every day, fast feedback on their work, fair grading, a credential at the end they can show recruiters.
- **Doesn't want:** more dashboards to babysit, generic AI chat that doesn't know them, exposure of their weaknesses to peers.

P2 • Mentor / Instructor

- Reviews candidate work and runs sessions. Has 20–60 candidates concurrently across a cohort.
- **Wants:** to know who needs them most today, automated first-pass review on routine work.

- **Doesn't want:** a tool that asks for more data entry, or an AI black box that overrides their judgement.

P3 · Training Manager / Admin

- Owns the cohort's outcomes — completion rate, average competency, placement readiness.
- **Wants:** predict who'll drop off, intervene before week 8, prove placement readiness to their client.
- **Doesn't want:** false at-risk flags that erode trust, an analytics tool that just produces charts no one acts on.

P4 · Recruiter (consumes credentials, not a system user)

- External — sees a candidate's credential URL on LinkedIn or resume.
- **Wants:** one click to verify the badge is real.
- **Doesn't want:** an LMS account or a phone call to "talk to the training provider."

4. The TPI Score

Every feature in this product ties back to the **Trainee Performance Intelligence (TPI) score**. Without understanding it, the rest of the document doesn't land.

The TPI score is a single **0–100 number** updated weekly per candidate. It's a weighted average of five dimensions:

DIMENSION	WHAT IT MEASURES	EXAMPLES OF EVIDENCE
Discipline	Did the candidate show up, on time, during work hours?	Login times, session frequency, daily activity within 9 AM–6 PM window
Learning	Are they understanding the content?	Quiz scores, quiz retake patterns, module completion rate
Technical	Is the code they're writing good?	Test pass rates, code quality from AI review, GitHub commit quality
Engagement	Are they actually consuming the material?	Video watch time, live session attendance, in-LMS active time
Growth	Are they improving over the program?	Week-over-week deltas across the four dimensions above

The composite TPI score maps to a level:

LEVEL	COMPOSITE RANGE	WHAT IT MEANS FOR THE CANDIDATE
STAR_PERFORMER	≥ 85	On track to be a top placement
ON_TRACK	65–84	Doing well, no concerns
NEEDS_ATTENTION	40–64	One or more dimensions soft — coaching needed
AT_RISK	< 40	Drop-off risk — intervention required

The TPI score is the **shared vocabulary** the candidate, the mentor, and the training manager all use to talk about progress. It's what the AI Tutor reads when deciding what to recommend. It's what the credential signs at the end. It's what the predictive engine ranks on. Everything ladders up to this number.

5. Feature: Multi-Agent TPI Scoring

Why this exists

Training programs have always run "blind" on whether a candidate is actually on track until it's too late. Weekly check-ins are subjective; final exams are too late. We needed an objective, behaviourally-grounded score that updates **automatically** every week so admins and candidates both know where they stand.

What it does

Five specialised AI agents — one per dimension — read each candidate's behavioural data and produce a 0–100 score. A sixth agent fuses them into the composite TPI. Every Sunday night the engine runs cohort-wide; by Monday morning, every admin dashboard and every candidate's "today's focus" reflects the new week.

Who uses it

It's **invisible infrastructure** for everyone — no user touches "the scoring engine" directly. What they see are the *outputs*: the TPI score on the dashboard, the level badge, the dimension breakdown.

Deep behaviour

- The Discipline agent only counts activity **during configured work hours** (default 9 AM–6 PM in the cohort's timezone). A candidate who's active at 3 AM doesn't get extra Discipline credit.
- The Learning agent doesn't just count quiz scores — it weights *retake patterns* (a candidate who scores 40% then retakes to 90% is learning; one who scores 40% three times is stuck).
- The Technical agent looks at both **in-LMS code submissions** and **GitHub commits** (see Feature 14).

- A failed agent for one candidate doesn't block the cohort — partial results persist with the failure logged.
- Candidates with no signal yet (week 1, or on documented leave) get a baseline from the cohort median, not a zero. Zero would feel punitive.

What success looks like

- $\geq 95\%$ of active candidates have a fresh score every week
- $< 5\%$ of candidates see their TPI swing more than 30 points week-over-week without a clear behavioural reason
- $< 2\%$ of scores get manually overridden by an admin — meaning the AI usually agrees with human judgement

Edge cases worth knowing

- **Brand-new candidate (week 1):** scored from cohort median until they have 14+ days of their own data.
- **Documented leave (illness, family):** admin flags it, scoring pauses, no penalty.
- **Zero activity for the week:** previous week's score $\times 0.85$ (gentle decay), flagged as `low_signal` so admins know it's not a fresh measurement.

What's deliberately out of scope (today)

- Real-time intraday scoring — weekly is enough; recomputing hourly is expensive and noisy.
- Per-candidate weight customisation — weights are cohort-level only.
- A machine-learning classifier (Phase 2 — current is rule-based with LLM narrative).

6. Feature: AI Personal Tutor

Why this exists

Candidates have questions every hour — "what's a hash map?", "why am I struggling with SQL?", "what should I do today?" — and there's one mentor per 30 candidates. The mentor gets the urgent stuff; the routine questions go unasked, which compounds confusion. The AI Tutor handles the routine 80% so the mentor gets the high-value 20%.

What it does

A **multi-turn chatbot** the candidate can ask anything course-related. Crucially, it **knows the candidate** — their name, their cohort, their current TPI breakdown, their recent weak quiz topics, whether they've linked GitHub, and what modules are in their course catalogue.

Two surfaces:

1. `/ai-tutor` — the full chat page with conversation history (rename, delete, return tomorrow).

2. **Dashboard widget "Today's focus"** — three concrete next-step recommendations refreshed daily, each a clickable card that lands on the right page (`/code-mentor` , `/dashboard/courses` , etc.).

Who uses it

Candidates — primary. The dashboard widget is the first AI surface a candidate sees every morning; the chat is what they fall back to when stuck.

Deep behaviour

- Every time the candidate sends a message, the tutor **re-reads their TPI state**. If their score just updated overnight, the tutor knows. Multi-day conversations stay current.
- The tutor is **forbidden from inventing modules**. If the cohort's course catalogue doesn't have a module on "advanced Rust," the tutor explicitly says so rather than fabricating one.
- It's **honest, not flattering**. If a candidate is at risk, the tutor says so gently — never pretends everything's fine.
- The tutor knows the **persona shift**. A candidate asking "what is recursion?" gets a tutorial; a candidate asking "what should I study?" gets a recommendation grounded in their weak topics.

What success looks like

- $\geq 50\%$ of active candidates use the tutor in any given week
- Average conversation has ≥ 4 messages (so it's not just one-shot Q&A)
- $\geq 30\%$ of the "today's focus" tiles get clicked

Edge cases worth knowing

- **Candidate has no TPI score yet (week 1)** — tutor falls back to cohort context + course catalogue only, says "I don't have your score yet, but here's where most candidates start in week 1."
- **Course catalogue empty** — tutor says "I don't have your module list yet" rather than inventing modules. Honesty > confabulation.
- **LLM call fails mid-conversation** — the candidate's message is still saved; they see a "tutor is briefly unavailable" message and can retry without losing context.

What's deliberately out of scope

- Voice input (Phase 2)
- Streaming the LLM response token-by-token (Phase 2 — currently waits for the full reply)
- File / code attachments in chat (Phase 2)
- Retrieval-augmented generation over the actual lesson video transcripts (Phase 3 — needs the embeddings pipeline)

7. Feature: AI Code Mentor

Why this exists

Code review is the highest-value mentor activity, but it's also the most rationed. A mentor of 30 candidates can do **maybe four code reviews per candidate per cohort** — not enough. The Code Mentor handles the routine "did I get this right?" check so the human mentor can focus on architectural feedback.

What it does

Three modes that live inside the in-LMS code editor:

MODE	WHEN THE CANDIDATE USES IT	WHAT THEY GET BACK
Hint	They're stuck, want a nudge	One Socratic question that references their specific code, never reveals the solution. <60 words.
Explain	They wrote code they don't fully understand	Plain-English walkthrough of what their code does + any bugs / smells. <150 words.
Pre-submission Review	Ready to submit, want a quality check	Structured Strengths / Improvements / Verdict (<code>ready</code> / <code>needs revision</code> / <code>blocked</code>). Catches syntax + correctness + performance + style, in that order.

Who uses it

Candidates — primary. The Pre-submission Review verdict becomes a quality gate they self-impose before clicking the LMS submit button.

Deep behaviour

- The candidate's code is **sent to the AI with line numbers attached** ("`1 | def two_sum(...)\n2 | ...`"). This eliminates the most common LLM failure: the model hallucinating tokens not actually in the buffer. The model can only cite lines it sees.
- The Review walk-through is **strictly ordered**: syntax & completeness first, then correctness, then performance, then style. If line 7 is a bare `return` (no value), verdict goes straight to `blocked`. Style nitpicks never appear before "your code has a syntax error."
- The mentor **doesn't auto-apply fixes**. It coaches, it doesn't autocomplete. This is deliberate — we want candidates to understand the why, not become dependent on AI-suggested patches.
- All three modes follow the same **grounding rule**: "treat the line-numbered source as truth, never invent code not on the page."

What success looks like

- $\geq 60\%$ of candidates use Code Mentor at least once per week
- Review verdict distribution settles around 40% `ready` / 45% `needs revision` / 15% `blocked` (means the AI is calibrated, not rubber-stamping)

- < 3% hallucination escape rate on manual audit (AI cited code that wasn't in the buffer)

Edge cases worth knowing

- **Empty buffer** — AI says "buffer is empty — write something first" instead of inventing code.
- **Non-code content** (e.g., a candidate pasted a README) — AI identifies it and asks the candidate to confirm intent.
- **Language we haven't seen** (e.g., Rust) — passed through; the underlying model handles most languages well enough.
- **AI is offline** — endpoint returns a graceful "AI mentor briefly unavailable" message; the candidate can keep coding.

What's deliberately out of scope

- Auto-applying fixes (we coach, we don't autocomplete)
- Running tests against the candidate's code (Phase 3 — needs sandbox infra)
- Auto-blocking the candidate's actual submission when verdict is `blocked` — currently advisory only

8. Feature: AI Mock Interview

Why this exists

Real interviews are high-stakes and infrequent. Candidates need volume practice — 5–10 mock interviews across topics — but mentors can't run that many 1:1 sessions. The AI Mock Interview gives every candidate unlimited practice runs, scored, with growth tracked over time.

What it does

The candidate picks **role × topic × difficulty** (e.g., "backend × Python × mid"). The AI conducts a structured interview of up to **6 turns**:

1. Opening question generated by the AI interviewer, calibrated to the candidate
2. Each turn: candidate types their answer → AI scores it 0–100 + writes per-turn feedback → AI generates the next question building on what they said
3. On finish: AI synthesises a markdown report (Strengths, Areas to Improve, Hire Signal) and computes per-dimension TPI impacts

Who uses it

Candidates — primary. Training managers see aggregate stats (who's practising, what scores) on an admin view.

Deep behaviour

- The interviewer **calibrates to the candidate's real TPI**, not just the difficulty they picked. A candidate who picks "senior" but has a TPI of 45 gets dialled-back questions — still challenging, not crushing. Honest scoring, but no humiliation.
- Each completed interview **feeds back into the TPI**. If a candidate scored well on a Technical interview, the Technical dimension gets a positive delta. The deltas show up the next time their weekly TPI runs.
- The history page shows a **growth chart** once they've done 2+ interviews on the same topic — "your Python interview score went 45 → 67 → 82." That's the dopamine that drives repeat practice.
- Stale sessions (left open > 2 hours) **auto-abandon** on next read. No fake "in progress" rows clogging the history.
- The AI is **honest in scoring**. A weak answer scores 30–50; an excellent one 80–95. >95 is reserved for exceptional depth. A junior candidate giving a junior-quality answer isn't penalised against senior expectations.

What success looks like

- $\geq 70\%$ of cohort completes at least one interview by week 8
- Average score-trajectory across 3+ sessions on the same topic shows positive slope $\geq +5$ points per session
- $\geq 75\%$ completion rate (started → finished, not abandoned)
- 10–25 minutes per session — long enough to be real, short enough to fit in a lunch break

Edge cases worth knowing

- **Candidate closes tab mid-session** — stale sweep auto-marks abandoned after 2 hours.
- **Candidate tries to submit two answers without waiting** — service rejects with "current turn already answered."
- **AI returns unparseable verdict** — falls back to a default `{score: 50, next_question: "Tell me more"}`. Session never deadlocks.

What's deliberately out of scope

- **Voice input** — text-only in v1. Speech-to-text (Whisper) is Phase 2 — adds maybe 5–6 hours of work but voice changes the product feel meaningfully so we want to ship it deliberately.
 - Pause/resume across days (Phase 2)
 - Live multi-interviewer panel ("HR + technical + manager") — Phase 3
-

9. Feature: AI Project Reviewer

Why this exists

Capstone reviews are **the** bottleneck of every cohort's last week. Mentors spend hours per project, candidates wait days for feedback. The AI Project Reviewer gives a structured first-pass verdict in under a minute, so the human mentor walks into the review already knowing the AI's score and can focus on the disputed parts.

What it does

The candidate submits a **GitHub repo URL** (typically their capstone). The reviewer:

1. Fetches repo metadata + README via the GitHub public API (no local clone needed)
2. Sends the README and metadata to the AI with a 5-dimension rubric
3. Produces a markdown report: Strengths, Gaps, Suggested Improvements, Verdict
4. Persists the row so the candidate (and admin) can see it later

The 5 rubric dimensions: **functionality, code quality, README quality, tests, deployment.**

Who uses it

- **Candidates** — submit projects, get feedback before the human review
- **Mentors** — read the AI's verdict before walking into the human review
- **Training managers** — see at-a-glance which projects passed AI review (admin dashboard)

Deep behaviour

- The review is **async**. Submission returns instantly with status `pending` ; the candidate's detail page polls every 3 seconds until it flips to `reviewed` . Typical latency: 20–60 seconds.
- The reviewer **handles GitHub redirects** — if a candidate renamed their repo since submitting, we follow.
- A **failed review** (private repo, GitHub down, AI returns garbage) sets status `failed` and stores the actual error in the feedback field. The candidate sees a real reason, not a stuck spinner.
- **Resubmissions are new rows**, not edits. The candidate's progression "v1: 62 → v2: 78 → v3: 91" is preserved as the audit trail.

What success looks like

- $\geq 80\%$ of cohort has at least one reviewed submission by week 12
- Mean review latency < 60 seconds
- $< 5\%$ non-recoverable failure rate
- $\geq 70\%$ human-AI agreement on overall verdict (when a mentor also reviews the same repo)

Edge cases worth knowing

- **Private repo** — fetch fails; status `failed` with explanation. Candidate either makes it public or gives the bot collaborator access.
- **No README** — AI scores `README quality` low and explicitly notes it. Doesn't blame the candidate for other dimensions.
- **AI returns invalid JSON** — we store the raw output anyway so the candidate sees *what* the AI said, even if our parser couldn't structure it.

What's deliberately out of scope

- **Actually running the code / executing tests** — needs sandbox infra (Phase 2 — Docker per submission, time-limited)
 - **Loom video analysis** — we capture the URL but don't process the video yet
 - **Linking resubmissions** — Phase 2 will add a "v2 of submission X" parent pointer
 - **Per-cohort rubric customisation** — Phase 2
-

10. Feature: Cohort Whisperer (Admin AI)

Why this exists

Training managers have to ask three questions every week: *who's at risk?*, *what's the common pattern?*, *what should I do about it?* — and the answer is buried in dashboards that take 20 minutes to navigate. The Whisperer turns those questions into natural-language chat backed by the real cohort data.

What it does

An admin types a question — "Who's most at risk this week and why?" — and the AI responds with **specific candidate names and concrete numbers** pulled from the cohort's TPI state. Not a generic answer. Real trainees.

Behind the scenes, every request loads a fresh snapshot of the cohort: top 5 + bottom 5 performers, 15 latest open alerts, week-over-week deltas, average TPI. That snapshot becomes the AI's grounding — it can only talk about what's in front of it.

Who uses it

Training managers and HR managers — admin-only. Mentors can be granted access on demand.

Deep behaviour

- The AI is **forbidden from inventing trainees, scores, or events**. The system prompt explicitly says: "Never invent names that aren't in the data."
- It **cites by name and number**. "Aanya Kumar dropped 12 points this week" — not "one of your candidates is declining."

- It's **honest about gaps**. If the admin asks "compare to last cohort" and we didn't load that data, the AI says "I don't have last cohort's data loaded — try the Reports view."
- The response includes a **transparency line**: "I looked at 3 trainees, 8 open alerts, avg TPI 67.8." So the admin sees what data slice fed the answer.

What success looks like

- $\geq 80\%$ of admin users ask at least one question per week
- Admin-reported "wrong answer" rate $< 10\%$ (qualitative — collected via in-app feedback)
- Average answer length 80–300 words (concise, not rambling)

Edge cases worth knowing

- **Cohort has zero scored trainees** — AI says so explicitly rather than inventing data.
- **Admin asks about something outside the loaded snapshot** (e.g., a specific assignment) — AI acknowledges the limit and suggests where to look.
- **AI generates a long response** — capped server-side, but the underlying prompt is tuned for conciseness so this rarely fires.

What's deliberately out of scope

- **Cross-cohort comparison** in a single prompt — one cohort at a time (Phase 2)
- **Action-taking from chat** ("schedule a 1:1 with X") — Phase 3
- **Voice queries** — Phase 3

11. Feature: Predictive Drop-off + Auto-Intervention

Why this exists

By the time a candidate visibly disengages — stops pushing, stops attending — the training budget for them is mostly spent. The product needs to **catch the trajectory** before the visible drop. And catching it isn't enough: the admin needs a one-click path to *do something about it*.

What it does

Two surfaces:

1. **Risk ranking** — every candidate scored 0–100 on drop-off likelihood, displayed as a ranked list per cohort. Each row shows the *primary factor* in plain English ("TPI dropped 12 this week", "no activity for 4 days") and a *suggested intervention* (`recovery_plan` / `mentor_pairing` / `extra_session`).
2. **One-click intervene** — admin clicks "Intervene"; the system dispatches an LLM-composed recovery plan to the candidate's notification inbox immediately.

Who uses it

Training managers (read + intervene), mentors (read-only).

Deep behaviour

- Risk score is computed from **four signals**, each weighted:
 - Distance from healthy TPI (`100 - latest_tpi`) — half-weight
 - Open alerts in the last 7 days — moderate weight, capped
 - Days idle since last activity — moderate weight, capped
 - Negative week-over-week delta — amplifies the score
- A trainee with **zero concerning signals returns 0**, with primary factor "no concerning signals." We don't fabricate risk where there isn't any.
- The intervention dispatch goes through the **Smart Notifications** dispatcher (Feature 12) — the LLM composes the actual nudge body based on the candidate's name, intervention type, and any admin-supplied context note.
- "Intervene all" is a one-click option for bulk dispatch — useful when a whole cohort hit a hard module and 5 trainees stumbled simultaneously.

What success looks like

- $AUC \geq 0.75$ on drop-off prediction at 4 weeks out (Phase 2 — once the ML classifier replaces the heuristic)
- $\geq 40\%$ of dispatched interventions produce candidate behavioural change within 7 days
- $< 25\%$ false positive rate (flagged as at-risk but recovered without intervention)

Edge cases worth knowing

- **Trainee with no GitHub linked, no quiz attempts, week 1** — risk score reads "no concerning signals" because we don't have enough data; doesn't fire a false alarm.
- **Same trainee intervened twice in an hour** — allowed but logged; Phase 2 will dedupe.
- **Notification dispatch fails** — the intervention row still returns; admin sees an error in the UI rather than silent failure.

What's deliberately out of scope

- **Real ML classifier** — currently rule-based; Phase 2 trains a model on accumulated cohort outcome data
 - **Auto-intervene on schedule** without admin click — deliberately gated to keep admin in the loop
 - **Per-trainee intervention history** view — Phase 2
-

12. Feature: Smart Notifications

Why this exists

A training program has dozens of nudges to send a candidate over 12 weeks: streak-broken, congratulations on improvement, recovery plan, recommended next module, reminder of upcoming session. Without a unified system, half of these land as one-off emails that get ignored. Notifications are the **product's voice** to the candidate; they need to be consistent, personalised, and measurable.

What it does

A **single inbox per candidate**, fed by every other AI feature in the product:

SOURCE	WHAT IT SENDS
AI Personal Tutor	Recommended next module
Predictive Drop-off	Recovery plan after intervention
AI Mock Interview	"You finished a session — here are your next steps"
Admin (manual)	Custom nudge to a specific trainee
Cron (Phase 2)	Streak reminders, weekly digest, milestone congratulations

For each notification, the caller picks a **category** + **tone** + optional context. The LLM composes the actual title and body. The candidate sees a tone-coloured card; clicks "read"; thumbs-up or thumbs-down for feedback.

Who uses it

- **Candidates** — read, act, give feedback (the inbox lives at `/notifications`, the unread count is the bell-icon badge in the header)
- **Admins** — compose new nudges manually; see delivery stats across the cohort

Deep behaviour

- The LLM **composes from a tone + category + candidate state**. Tone vocab: `encouraging` / `firm` / `friendly` / `urgent`. Category vocab: `streak` / `quiz` / `engagement` / `intervention` / `recommendation` / `congratulations` / `announcement`.
- Caller can override title + body manually if they want. The LLM only fires when at least one is null.
- The candidate inbox shows **tone-coloured left borders** so urgency is visually distinguishable at a glance.
- Every notification has an optional **action URL + action label** (e.g., "Open recovery plan" → `/ai-tutor`). The system tracks `acted_at` separately from `read_at` so we measure intent vs. just-saw-it.

- The thumbs-up/down feedback is **persisted per notification** and fed back into future LLM prompts ("the candidate found similar nudges helpful").

What success looks like

- 7-day read rate $\geq 70\%$
- 7-day action rate $\geq 25\%$ (clicked the call-to-action)
- Thumbs-up : thumbs-down ratio $\geq 5:1$
- Unsubscribe rate $< 3\%$ (Phase 2)

Edge cases worth knowing

- **LLM returns malformed title/body JSON** — falls back to a category-string title + empty body. Candidate sees something, never an exception. Phase 2 will improve the prompt to make this rare.
- **Candidate opens inbox before any notification exists** — friendly empty state with a copy hook: "Your AI mentor's nudges will appear here."
- **Action URL points to a deleted resource** — frontend handles the 404 naturally; we don't pre-validate every URL.

What's deliberately out of scope

- **SMS / email / push channels** (Phase 2 — schema has a `channel` column ready)
- **Quiet hours / Do Not Disturb** per candidate (Phase 2)
- **Notification templates editor** for admins to write reusable patterns (Phase 3)

13. Feature: Verifiable TPI Credential

Why this exists

Bootcamps say "94% placement rate" and nobody outside the bootcamp world believes them. There's no portable, tamper-proof record of what a graduate actually achieved. Recruiters either trust the bootcamp's brand or they don't. The Verifiable TPI Credential turns "trust me" into "verify it yourself in one click."

What it does

At graduation, an admin **issues** a credential for a candidate. The system signs a JWT (HS256) over the candidate's final TPI score, dimension breakdown, cohort name, and issue date. The candidate gets a shareable verification URL and a QR code.

Recruiters (or anyone) can hit the public verification URL — **no LMS login required** — and the system returns a green checkmark + the badge data, or a red X with a reason (signature invalid / revoked / not found).

The candidate's `/credential` page in the user app shows:

- The signed badge (TPI score, level, dimension breakdown)
- A QR code linking to the public verification URL
- A "Copy verification URL" button
- A "Share to LinkedIn" button

Who uses it

- **Candidates** — own the badge, share it
- **Admins** — issue, revoke, see verification logs
- **Recruiters** — verify

Deep behaviour

- The credential **is** the JWT. The database row exists only to support revocation lookups and audit logging. Even if the database disappears, the JWT signature still validates as long as the secret is intact.
- **Revocation doesn't delete the row** — it sets `revoked_at`. The historical record stays. A recruiter who verifies later gets "this credential was revoked at [date]" rather than "not found."
- The QR code on the candidate's page is generated client-side from the public verification URL. Scannable with any phone camera.
- **The recruiter never sees the LMS**. The verification URL is the only surface they interact with. This is deliberate — we don't want to bleed into LMS branding when recruiters consume credentials.

What success looks like

- 100% of graduating candidates have an issued credential
- ≥ 2 share clicks per credential per month (LinkedIn + direct share)
- ≥ 1 recruiter verification hit per credential per month
- Forgery attempts (invalid signature submissions) are tracked but un-targeted — just monitored

Edge cases worth knowing

- **Candidate has no TPI score yet** — issuance fails with "no TPI scores yet" — we can't issue an empty badge.
- **JWT signature invalid** — verification returns `{valid: false, reason: "Signature invalid: ..."}`
- **Credential ID in JWT doesn't exist in DB** (someone forged a JWT) — verification returns "Credential not found"
- **Credential was revoked** — verification returns "Credential revoked at [timestamp]"

What's deliberately out of scope

- **Asymmetric signing (RS256)** with a public verification key — Phase 4. HS256 is fine for in-network verification; for true external trust we'll publish a public key.

- **Blockchain anchoring** — Phase 4, gas-free chain like Polygon
 - **Credential transfer to another identity** — never. The credential is bound to the candidate; transferring it would defeat the point.
 - **Auto-issue at cohort end** — currently admin-triggered. Phase 2 adds a Celery task that auto-issues for all `STAR_PERFORMER` / `ON_TRACK` graduates when their cohort closes.
-

14. Feature: Daily Commit Cadence & Code Quality as a Signal

Why this exists

Coding is what the candidate is being trained to do. Their commit history — how often they push, what they push, how their code quality trends — is the **richest behavioural signal we have**. The "did they show up today and do real work?" question is answered most truthfully by `git log`, not by quiz timestamps.

The Discipline dimension wants to know: are they working **every day**? The Technical dimension wants to know: is the code **getting better**? GitHub gives us both, but only if we capture the signal cleanly.

What it does

Every candidate links their personal GitHub account once, via the candidate-side OAuth flow at `/profile/integrations`. From that moment:

1. Every push to their personal cohort repo flows back to us via a webhook
2. Every commit is timestamped, weighted by **time-of-day** (work-hours pushes count more than 3 AM pushes), and counted toward Discipline
3. Each push triggers an AI code-quality pass over the diff — the result feeds the Technical dimension
4. The Discipline agent specifically looks at **streak**, **commits-per-day average**, and **work-hours adherence** as separate sub-signals

The candidate sees their commit cadence on their own dashboard. The mentor sees it on the trainee detail page. The training manager sees cohort-wide cadence in the dashboard analytics.

Who uses it

- **Candidate** — their commit graph becomes part of their portfolio narrative ("look how consistently I worked")
- **Mentor** — early warning that someone hasn't pushed in 5 days, before TPI drops
- **Training manager** — cohort-wide "did this cohort show up this week?" indicator

The repo strategy (the question this section was built to answer)

Per-candidate personal repos, not a shared cohort mono-repo. Each candidate has their own GitHub repo (forked from the cohort's template) where they push all their daily training-program

code.

Why per-candidate, not shared:

CONSIDERATION	PER-CANDIDATE REPO	SHARED COHORT MONO-REPO
Privacy of solutions	✅ Candidate A can't see Candidate B's work	❌ Everyone reads everyone — defeats the point of practice
Commit-streak signal	✅ Clean per-candidate <code>git log</code>	❌ Cohort log is 30× noisier, harder to isolate
Portfolio at graduation	✅ Candidate can show "my repo, 200 commits, 12 weeks" to recruiters	❌ "I was one of 30 contributors" — much weaker placement story
Webhook routing	✅ One push = one candidate = one routing decision	❌ Have to parse file paths to figure out <i>which</i> candidate pushed
Permissions	✅ Candidate owns their repo, can make it public for portfolio	❌ Git has no per-folder push permission; either everyone can push everywhere or PR-only — both bad for daily practice
Cohort history	✅ Last cohort's repos stay in candidates' accounts; new cohort starts fresh	❌ History fragments — do you reuse the mono-repo or make a new one? Either way it's awkward.

The repo structure is:

```
github.com/<customer-org>/cohort-2026-A-template      (public; the starter)
github.com/<customer-org>/cohort-2026-A-answer-keys    (private; instructor-only)

github.com/<candidate>/zennial-2026-A                 (their fork; daily work)
github.com/<candidate>/zennial-capstone-<project>       (separate repo; capstone)
```

Capstones get a **separate repo per candidate** — they have their own README, deploy config, and recruiter-facing URL. We don't bury the capstone inside the cohort fork.

Deep behaviour

- **Commit cadence > commit volume.** A candidate pushing once a day, 5 days a week is rewarded more than one pushing 50 commits on Friday night. The Discipline agent counts *distinct days with at least one commit during work hours*.
- **Work-hours weighting:** a push at 11 AM counts; the same push at 11 PM counts much less. The cohort's work-hours window is configurable per cohort (default 9 AM–6 PM in the cohort's timezone).
- **Quality, not just quantity.** Pure-volume pushers ("I pushed `print('hi')` 30 times") don't game the system — every commit goes through an AI code-quality pass that scores the diff. A

high-volume / low-quality candidate ends up with a strong Discipline score but a weak Technical score, and the AI Tutor recommends accordingly.

- **Streak rewards but doesn't punish.** Missing a day doesn't tank your score; the agent treats it as a soft signal. But a 5-day idle streak triggers a notification ("haven't seen a push from you this week — everything OK?").
- **Catalog matches reality.** Folders inside the candidate's repo mirror the cohort template (`week-01-fundamentals/` , `week-02-data-structures/` , etc.). The AI Code Mentor (Feature 7) and the AI Tutor (Feature 6) both use folder names to ground their recommendations.
- **Ownership at graduation.** The fork stays in the candidate's GitHub account forever. Zennial Pro doesn't delete or rename it after the cohort ends. The candidate keeps their portfolio.

Onboarding — Day 1 of a cohort

1. Admin sets up the cohort template repo once in the customer's GitHub org
2. Candidate links GitHub via `/profile/integrations` (existing OAuth flow)
3. Behind the scenes, the LMS forks the template into the candidate's account, registers the fork URL, installs the webhook
4. Candidate clones their fork, starts pushing
5. Every push → webhook → cadence + quality scoring → feeds Discipline + Technical TPI dimensions

The whole onboarding takes the candidate ~2 minutes. They never see the GitHub API calls.

What success looks like

- $\geq 95\%$ of enrolled candidates have a personal cohort fork by end of week 1
- $\geq 80\%$ of forks are emitting webhook events by end of week 2 (i.e., they're actually using it, not just linked)
- Webhook delivery success rate $\geq 99\%$
- Mean time from `git push` to TPI scoring event recorded < 30 seconds
- $\geq 70\%$ of candidates retain their repo as a portfolio piece 3 months post-graduation

Edge cases worth knowing

- **Candidate makes their fork private** — webhook still works (it's installed on the repo, not dependent on visibility). The AI code-review service uses an authenticated token if it needs to read code.
- **Candidate name collision** — if `zennial-2026-A` already exists in their GitHub account from a previous cohort, the new fork gets a numbered suffix (`zennial-2026-A-2`). We record the actual fork name.
- **Candidate disconnects GitHub mid-cohort** — webhook events stop being routable; the link row stays but is flagged `disconnected` . Re-linking re-enables.
- **Customer rotates the cohort template mid-cohort** — existing candidate forks don't auto-rebase. Admin can trigger a "sync" task or instruct candidates to manually merge. Auto-sync

is Phase 2.

- **Force push** — the webhook still fires; TPI score reflects what's on the remote at HEAD now. Historical events from before the force push stay in the event log; nothing's deleted.
- **Candidate removed from cohort** — fork stays in their account (they keep ownership); the link row is marked `archived` so no further scoring.

The deliberately rejected alternative

We considered a **single shared cohort mono-repo with per-candidate folders**:

```
github.com/<customer-org>/cohort-2026-A/  
└─ candidates/  
    └─ aanya-kumar/  
    └─ ranjith-rayapati/  
    └─ ... (30 folders)
```

We rejected it for the reasons in the comparison table above. Three narrow exceptions where the mono-repo IS right (and we use it for those):

1. **Answer-keys archive** — single private repo, instructors only. By design shared.
2. **Live pair-programming sessions** — throwaway shared repo for the session, deleted after.
3. **Cohort-built shared infra** (e.g., a cohort builds a library together) — one repo because it IS one project.

For **graded individual work**: always per-candidate.

What's deliberately out of scope

- **Auto-sync of template changes** into existing candidate forks (Phase 2)
- **Per-week branch enforcement** ("you must work on the `week-3` branch this week") — too rigid, not worth the friction
- **PR-based mentor workflow** (Phase 2 — currently mentor reviews via direct repo access)
- **GitLab / Bitbucket support** — GitHub-only for v1
- **Migrating pre-Zennial candidates** whose code lives in non-standardised repos — out of scope; we work with whatever repo they link

15. Cross-feature loops

This is where the product becomes more than the sum of its features. Three loops that compound:

Loop 1 — The Mock-Interview ↔ TPI loop

```
Candidate completes a mock interview
→ AI synthesises a report
→ Score impact deltas written to TPI dimensions
→ Next week's TPI score reflects this interview
→ AI Tutor reads the updated TPI and recommends differently
```

Every interview improves the TPI's calibration of the candidate. Every TPI update changes what the tutor recommends next.

Loop 2 — The Commit ↔ Tutor ↔ Recommendation loop

```
Candidate pushes code at 11 PM (outside work hours)
→ Discipline dimension lightly dings
→ Risk score nudges up
→ Predictive engine flags them in "at-risk" list
→ Admin clicks "Intervene"
→ Notification dispatched ("hey, your push pattern shifted to late nights – everything")
→ Candidate either explains (admin notes it) or adjusts schedule
→ Next week's Discipline score recovers
```

The system catches a behavioural shift two weeks before the candidate would visibly disengage.

Loop 3 — The Code Mentor ↔ Project Review ↔ Credential loop

```
Candidate uses Code Mentor for daily problems
→ Pre-submission Review verdicts log to Technical dimension
→ AI Project Reviewer scores the capstone
→ Final TPI captures both daily-quality and capstone-quality
→ Credential signs the composite at graduation
→ Recruiter scans the credential, sees a real score backed by 12 weeks of behavioural evidence
```

The credential at the end isn't just "they passed the bootcamp" — it's "here's the actual evidence trail."

These loops are why a competitor can't replicate this by bolting "ChatGPT for students" onto their LMS. The data dependencies between features are the moat.

End of document.